Introduction to Database Systems

LECTURE 08

Vector Databases

From embeddings to similarity search and retrieval-augmented generation

Xiaofan Li School of Computer Science
Nanjing University

Review: Representation and Data Models

INFORMATION REPRESENTATION

How is information encoded?

Classical bit: either 0 or 1.

Qubit (previous lecture): a state $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$, with $|\alpha|^2 + |\beta|^2 = 1$.

DATA MODEL

How is data organized and queried?

Relations organize tuples into tables; graphs organize vertices and edges; vector retrieval compares numerical vectors by similarity.

Today's focus: represent meaning with vectors and retrieve similar items. These vectors are still stored as classical bits.

Relational Queries vs. Semantic Search

Data: `Document(id, text)` Example row: (1, “Scenic trails in national parks”)

User intent: “Where can I go for a hike?”

RELATIONAL QUERY: A TEXT PREDICATE

```
SELECT id, text FROM Document
WHERE text LIKE '%hike%';
```

This row is excluded: its text does not contain “hike”. The predicate is false.

VECTOR QUERY: SEMANTIC SIMILARITY

Embed the query and documents; return the top-k closest document vectors.

This row may rank highly: “trails” and “hike” have related meanings.

Relational DBMSs can also support vector search through extensions.

Lecture Roadmap

01 **From Data to Similarity Search**

Embeddings, similarity measures, and exact nearest neighbors

02 **Graph-Based Approximate Search**

Navigate a proximity graph instead of scanning every vector

03 **HNSW: Search, Construction, and Tradeoffs**

Use a hierarchy; build the index; balance cost and recall

04 **Beyond Dense Vector Search**

Keywords, BM25, hybrid search, and multilingual retrieval

05 **Retrieval-Augmented Generation**

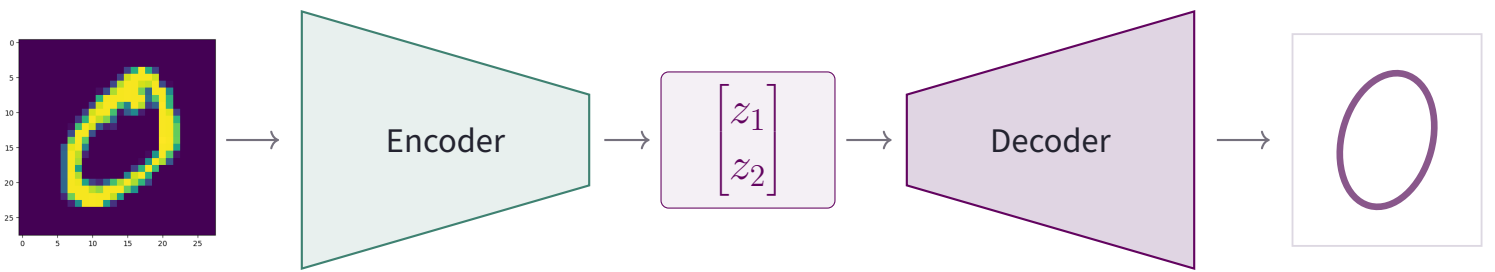
Retrieve evidence, construct the prompt, and generate an answer

CHAPTER 1

From Data to Similarity Search

Learning an Image Embedding

Compress an image into a small vector, then learn to reconstruct it.



Input image
$28 \times 28 = 784$

$784 \to 256 \to 128$

Embedding
2 dimensions

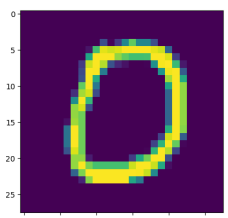
$128 \to 256 \to 784$

Reconstruction
(illustration)

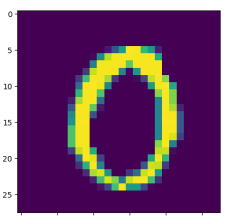
The decoder receives only the embedding. Training encourages reconstruction of the input; 2D is chosen for visualization.

Comparing Two Zeros and a One

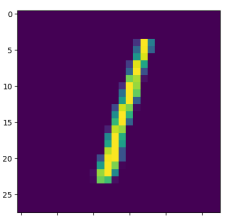
Encode three images with the **same model**, then compare their vectors.



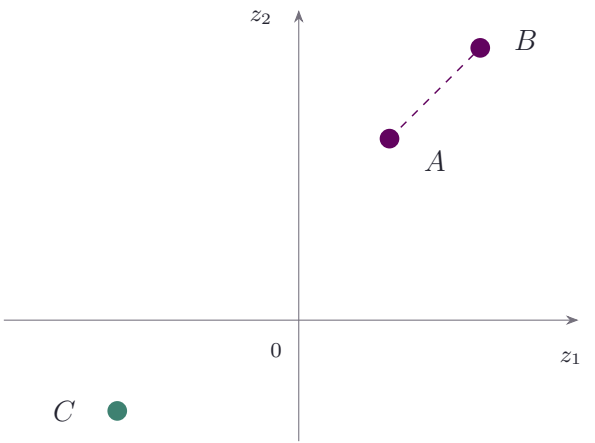
$A: (1, 2)$



$B: (2, 3)$



$C: (-2, -1)$



Different images of zero map to nearby points; the image of one lies farther away.

Illustrative 2D embeddings, chosen for hand calculation. Learned coordinates depend on the trained model.

Sentence Embeddings

A sentence encoder maps each text to a fixed-length vector.

ID	Sentence	Output
s_0	The team enjoyed the hike through the meadow	$v_0 \in \mathbb{R}^{384}$
s_1	The national park had great views	$v_1 \in \mathbb{R}^{384}$
s_2	Olive oil drizzled over pizza tastes delicious	$v_2 \in \mathbb{R}^{384}$

```
model = SentenceTransformer('paraphrase-MiniLM-L6-v2')
vectors = model.encode(sentences)
print(vectors.shape)  # (3, 384)
```

An embedding is a learned representation, not a lossless copy of the text or a list of named semantic attributes.

Euclidean and Manhattan Distance

Reference vectors: $A = (1, 2)$, $B = (2, 3)$, $C = (-2, -1)$. Smaller distance means closer.

EUCLIDEAN DISTANCE (L_2)

$$d_2(x, y) = \sqrt{\sum_{j=1}^{d} (x_j - y_j)^2}$$

Straight-line distance between two points.

$$d_2(A, B) = \sqrt{1^2 + 1^2} = \sqrt{2}$$

MANHATTAN DISTANCE (L_1)

$$d_1(x, y) = \sum_{j=1}^{d} |x_j - y_j|$$

Distance when moving along one axis at a time.

$$d_1(A, B) = |1| + |1| = 2$$

For comparison, $d_2(A, C) = 3\sqrt{2}$ and $d_1(A, C) = 6$: the one is farther from the first zero.

Dot Product Similarity

The dot product is a **similarity score**: retrieve larger scores first.

$$x \cdot y = \sum_{j=1}^{d} x_j y_j = \|x\|_2 \|y\|_2 \cos\theta$$

Pair	Calculation	Score
A, B	$1 \cdot 2 + 2 \cdot 3$	8
A, C	$1 \cdot (-2) + 2 \cdot (-1)$	-4

Direction and length both matter.

Doubling B doubles $A \cdot B$, even though B still points in the same direction.

$A = (1, 2)$, $B = (2, 3)$, $C = (-2, -1)$. Dot product is not a distance; a negative score means an obtuse angle.

Cosine Similarity and Cosine Distance

Cosine compares the **directions** of two nonzero vectors, ignoring their lengths.

SIMILARITY: LARGER IS CLOSER

$$\mathrm{sim}_{\cos}(x, y) = \frac{x \cdot y}{\|x\|_2 \|y\|_2}$$

DISTANCE: SMALLER IS CLOSER

$$d_{\cos}(x, y) = 1 - \mathrm{sim}_{\cos}(x, y)$$

For $A = (1, 2)$ and $B = (2, 3)$:
$\mathrm{sim}_{\cos}(A, B) = 8/\sqrt{65} \approx 0.9923$.

$d_{\cos}(A, B) \approx 0.0077$.
For $C = (-2, -1)$, $d_{\cos}(A, C) = 1.8$.

Cosine distance is $1 - \cos\theta$, not the angle θ itself. Identical directions give distance 0.

Comparing the Three Sentences

s_0: hike through a meadow s_1: national-park views s_2: olive oil on pizza

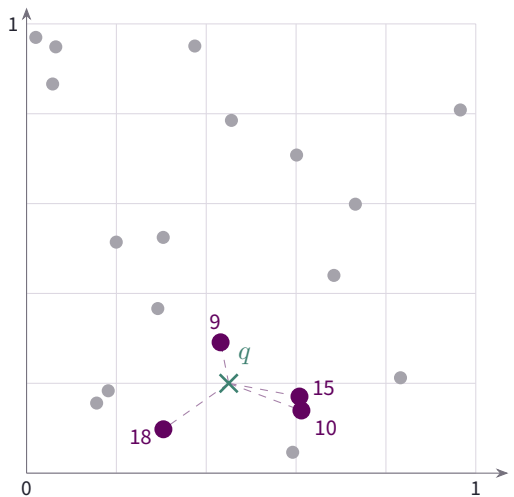
Pair	Dot product ↑	Cosine distance ↓	Interpretation
s_0, s_1	26.498	0.535	Most similar pair
s_0, s_2	2.079	0.964	Less similar
s_1, s_2	4.019	0.929	Less similar

The first two sentences share an outdoor theme, despite using different words. They are related, but do not express the same proposition.

Example outputs for `paraphrase-MiniLM-L6-v2`, rounded. Scores depend on the model and preprocessing.

Exact k-Nearest-Neighbor Search

Exact kNN returns the k closest stored vectors to a query. Here: 20 vectors, $q = (0.45, 0.20)$, $k = 4$.



1 SCORE Compute the distance to every vector.

2 RANK Sort by increasing distance.

3 RETURN Take the first k records: IDs and scores.

ID	Coordinates	Distance
9	$(0.4319, 0.2912)$	0.0930
15	$(0.6075, 0.1705)$	0.1603
10	$(0.6119, 0.1395)$	0.1728
18	$(0.3046, 0.0977)$	0.1778

Purple points are returned. IDs start at 0; values are rounded. For similarity scores, rank largest first.

Why Brute Force Becomes Expensive

Every query compares against N vectors, and each comparison processes d coordinates.

$$\underbrace{\Theta(Nd)}_{\text{compute all scores}} + \underbrace{O(N \log N)}_{\text{sort all scores}}$$

Vectors N	Dimensions d	$N \times d$	Coordinate pairs per query
20	2	40	Easy to inspect and calculate
$500{,}000$	768	$384{,}000{,}000$	Much more work for a single query

Top-k selection can avoid a full sort. The brute-force scan still scores every vector; ANN aims to reduce that work.

CHAPTER 2

Graph-Based Approximate Search

Approximate Nearest Neighbors

ANN searches a promising part of the dataset. It may miss an exact nearest neighbor in exchange for less work.

EXACT TOP 4

$\{9, 15, 10, 18\}$

ILLUSTRATIVE ANN TOP 4

$\{9, 15, 10, 14\}$

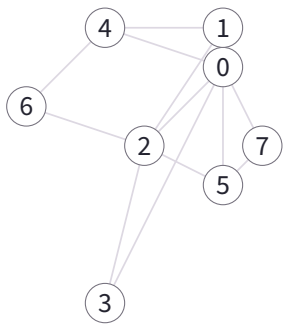
$$\text{Recall@4} = \frac{|A \cap E|}{4} = \frac{3}{4} = 0.75$$

E: exact neighbor IDs.
A: returned neighbor IDs.

Recall measures agreement with exact vector neighbors; semantic relevance also depends on the embeddings.

From Vectors to a Graph

Node: one stored vector. **Edge:** a connection to a selected neighbor.



New node	Vector	Connect to
2	$(5, 5)$	$0, 1$
6	$(2, 6)$	$4, 2$
7	$(8, 5)$	$5, 0$

PARAMETER: M

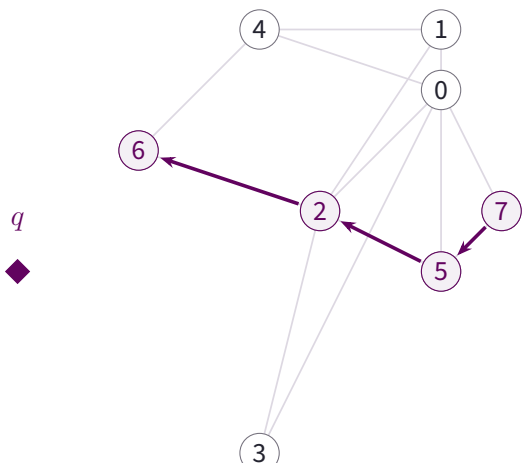
Select up to M earlier neighbors for each new node. We choose $M = 2$ for this example.

1. Insert vectors in order: $x_0, x_1, \ldots, x_7$.
2. Compute distances to all earlier vectors.
3. Select the M nearest earlier nodes.
4. Add edges both ways; repeat for the next node.

Euclidean distance is used here. If fewer than M earlier nodes exist, use all of them. Scalable NSW searches for candidates.

Greedy Search on the Graph

Query $q = (0, 4)$. Follow the closest neighbor while the distance improves.



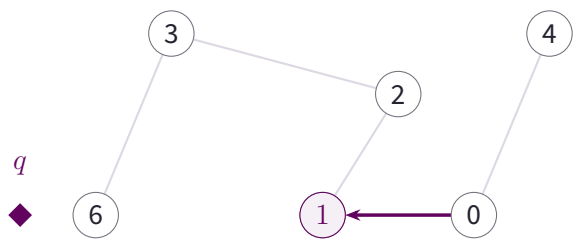
ONLY FOLLOW STORED LINKS

Node	Distance to q	Approx.
7	$\sqrt{65}$	8.06
5	7	7.00
2	$\sqrt{26}$	5.10
6	$\sqrt{8}$	2.83

Stop at 6: none of its neighbors is closer. In this example, 6 is also the exact nearest neighbor.

Why Greedy Search Can Miss the Best Match

A **local minimum** beats all its neighbors, but may not be the closest vector in the graph.



SEARCH STARTS AT NODE 0

$0 \to 1$ moves closer to the query.

At 1, both neighbors are farther away. Greedy search stops.

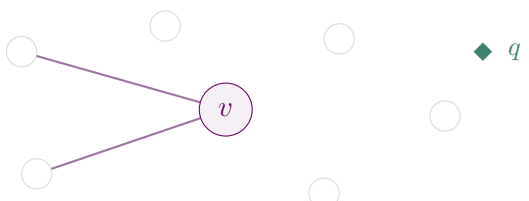
Yet 6 is closer. Reaching it requires the detour $1 \to 2 \to 3 \to 6$.

More useful links can reduce trapping, but add storage and neighbor checks. Wider search can explore alternatives.

Few Neighbors or Many: Why Connectivity Matters

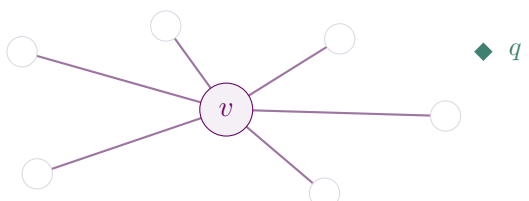
A node's **degree** is its number of neighbors. Greedy search can only compare the neighbors connected to its current node.

LOW DEGREE: FEWER CHOICES



Few available moves can make the search stop early at a local minimum.

HIGH DEGREE: MORE CHOICES



More routes may help, but more edges cost memory and neighbor checks.

NSW routing intuition: a search can progress from higher-degree nodes without increasing much time complexity.

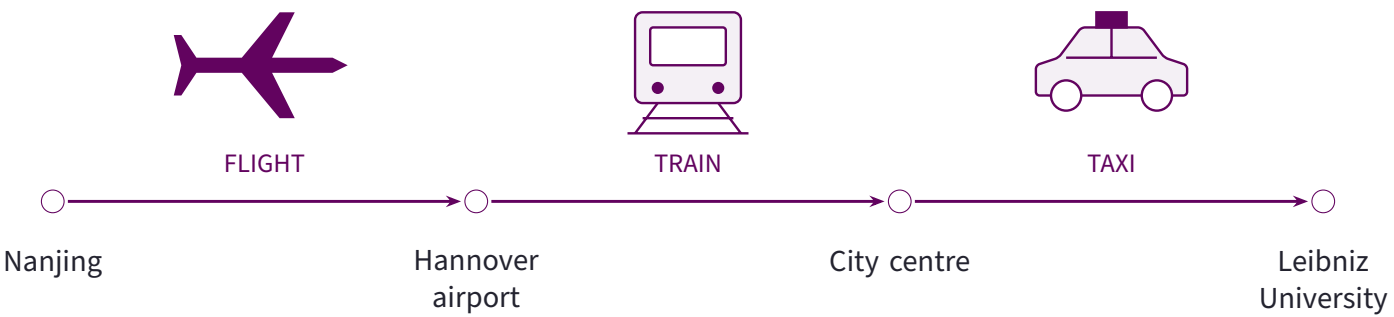
HNSW adds sparse upper layers for broad navigation, then refines below. High degree alone does not guarantee a good route.

CHAPTER 3

HNSW: Search, Construction, and Tradeoffs

From Nanjing to Leibniz University Hannover

Fly from Nanjing to Hannover, take a train into the city, then a taxi to the university. Each stage brings us closer at a **finer scale**; we do not take a taxi all the way from Nanjing.



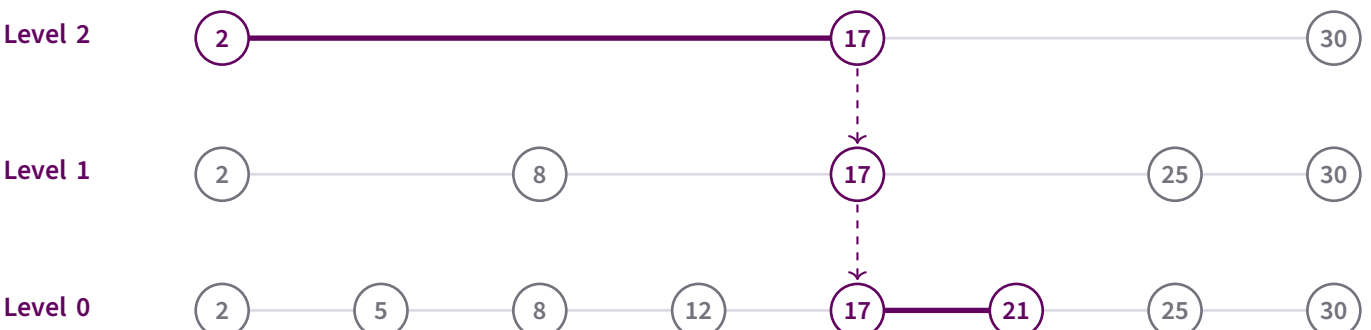
HNSW follows the same idea: first navigate a **sparse** upper layer to reach a promising region, then descend into **denser** layers to find nearby vectors.

Schematic journey; flights may include connections. Changing layers keeps the same vector as the starting point.

Skip Lists (1990): Structure and Search

Problem: Given a sorted linked list and a target key (here, 21), find the key or report that it is absent, using fewer comparisons than a sequential scan.

A **skip list** adds sparse upper levels to a sorted linked list, allowing search to skip intermediate keys. Randomly promoted keys remain in all lower levels.



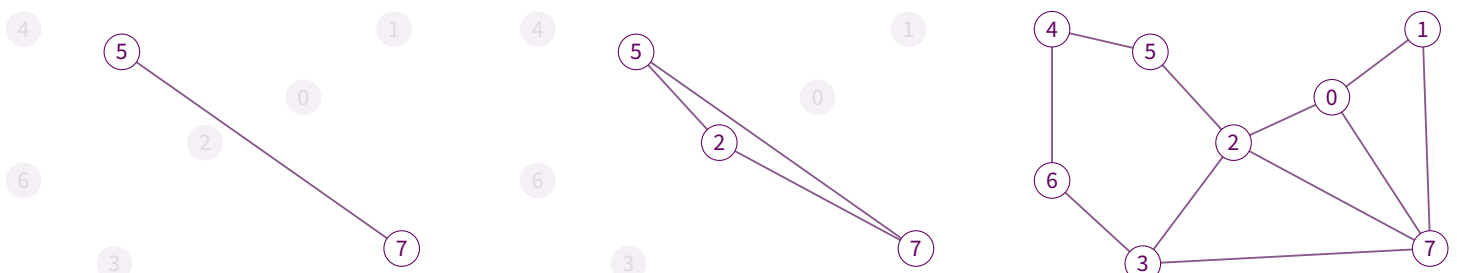
Find 21: move right while the next key is ≤ 21; otherwise move down. Stop at 21.

William Pugh (1990). Expected search: $O(\log N)$. HNSW borrows the hierarchy, but vectors have no single sorted order.

HNSW: A Hierarchy of Proximity Graphs

HNSW = Skip List's hierarchy + NSW's proximity-graph navigation.

LAYER 2 → **LAYER 1** → **LAYER 0**



Few nodes: broad navigation. More nodes: refine the region. All vectors: local search.

Same IDs and positions across layers. Pale nodes are absent from that layer. Descend at the same vector; edges stay within a layer.

Query Search: Navigate, Descend, Then Refine

Input: query vector q. **Output:** the k nearest vectors found.

`efSearch` $\geq k$: maximum retained result-set size in the bottom-layer search.

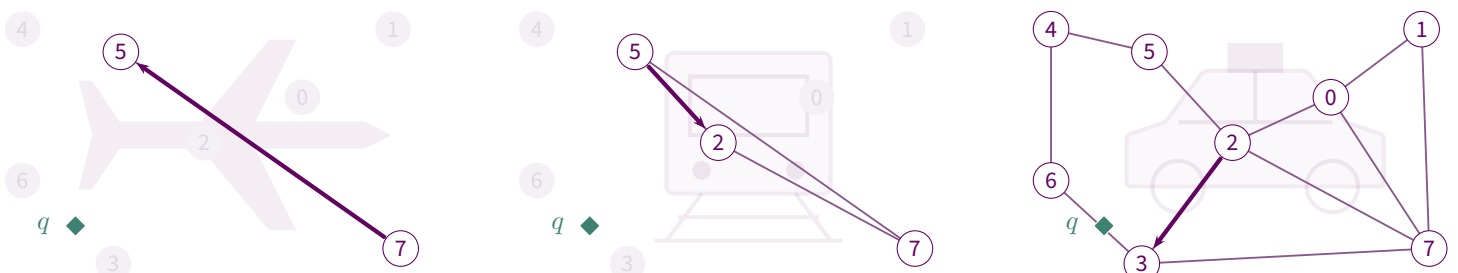
01	START HIGH	Begin at the entry point in the highest layer.
02	MOVE + DESCEND	Move to closer neighbors until none improves the distance. Descend at that same vector; repeat in each upper layer.
03	REFINE + RETURN	At layer 0, explore multiple candidates with `efSearch`. Return the best k.

Queries only read the graph. Insertion searches use the new vector as a query, then add graph connections.

HNSW Search: Move Closer, Then Move Down

Same graph as before. The green diamond is query q. Start at node 7 in layer 2; descend at the best node found.

LAYER 2 → **LAYER 1** → **LAYER 0**



$7 \to 5$; descend at 5.

$5 \to 2$; descend at 2.

$2 \to 3$; nearest to q.

Purple arrows show improving moves in this example. Query q is not inserted. Layer 0 may explore multiple candidates with efSearch.

Building the Index

Assign layers, find neighbors, and connect each new vector.

Build the Graph: Assign Each Vector Its Layers

Insert vectors **one at a time**. Randomly choose a **highest level** L for each vector; include it in layers $0, 1, \ldots, L$.

Vector	L	Present at
0	0	0
2	1	0, 1
5	2	0, 1, 2
6	0	0

SAME VECTOR, SAME COORDINATES

A node does not move in vector space when it is promoted. Only its membership in the layers changes.

Every vector belongs to level 0.

Insert a Vector: Search First, Then Connect

A new photo has vector x. Insert its ID, vector and links into the index so future searches can reach it. Its coordinates stay fixed. Suppose its sampled highest level is 1.

1. FIND THE RIGHT REGION

Use x as a query. Start at the top entry point, follow existing edges toward x, and descend. Layers 3 and 2 already have a graph; they do not receive x.

Layer 3: navigate existing edges

Layer 2: navigate existing edges

Layer 1: add x and neighbor links

Layer 0: add x and neighbor links

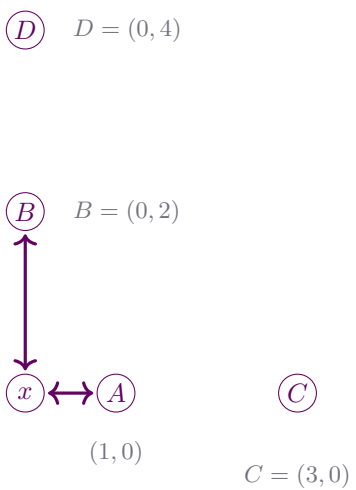
2. ADD THE NODE AND CONNECTIONS

In layer 1, find nearby candidates and select up to M neighbors (M is the connection limit). Add each selected neighbor to x's list, and x to theirs. Repeat in layer 0.

Upper layers help locate neighbors efficiently. Repeat for each new vector; the first vector initializes the index.

Insert a Vector: Select Neighbors and Add Links

Continue at layer 1. Search finds candidates A, B, C, D in this layer. For this basic example, select the nearest $M = 2$ candidates to $x = (0, 0)$.



1. COMPARE CANDIDATES

Node	Distance to x	Decision
A	1	Keep: closest.
B	2	Keep: second closest.
C	3	Not selected: farther away.
D	4	Not selected: farther away.

2. ADD LINKS IN BOTH DIRECTIONS

$x \leftrightarrow A$ and $x \leftrightarrow B$. Repeat at layer 0.

This example uses nearest-only selection. HNSW can refine it to preserve diverse routes. All new edges stay within the current layer.

Search Cost and Parameters

Understand the search cost, then balance recall, speed, and memory.

Sparse Upper Layers Give About $\log N$ Levels

N: stored vectors; p: chance of promotion to the next layer. A node's highest level is L; N_ℓ counts nodes in layer ℓ.

FEWER NODES AT EACH LEVEL

$$\Pr(L \geq \ell) = p^\ell, \qquad \mathbb{E}[N_\ell] = Np^\ell$$

Near the highest level h, about one node remains. Solve for h:

$$Np^h \approx 1 \quad \Rightarrow \quad h \approx \frac{\log N}{\log(1/p)}$$

EXAMPLE: $N = 10^6$, $p = 0.1$

Layer	Expected nodes
0	1,000,000
1	100,000
2	10,000
3	1,000
4	100
5	10
6	1

Taking logs gives $\log N + h \log p \approx 0$. For fixed p, $h = O(\log N)$: here about 7 layers (0–6). Actual counts and heights vary.

From Layer Count to Query Work

Count distance calculations first: every time we examine a neighbor, we compare its vector with the query.

WORK IN THE UPPER LAYERS

h: number of upper layers.
s: average nodes expanded per upper layer.
c: average neighbors checked per expansion.

upper-layer work $\approx hsc$

ADD THE BOTTOM-LAYER SEARCH

Let B be distance calculations at layer 0. It depends on the data and `efSearch`; it is not equal to `efSearch`.

total $\approx hsc + B$

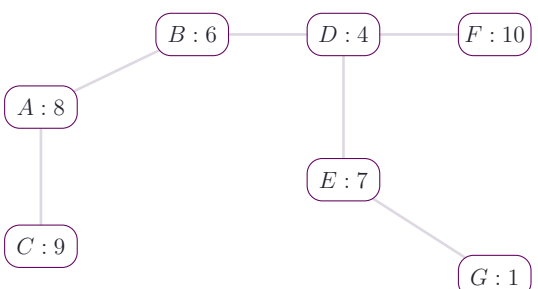
If s, c remain small and bottom-layer work is controlled, search can scale roughly as $\log N$. This is not a worst-case guarantee.

Querying: Keep Candidates, Then Return Answers

Find the $k = 1$ vector closest to query q. Start at entry node A. A label such as $A : 8$ means $d(q, A) = 8$; smaller distances are better.

`ef` is commonly called the **exploration factor**; `efSearch` controls the search shortlist capacity.

ONE LAYER OF THE EXISTING GRAPH



TWO DIFFERENT NUMBERS

`efSearch`:
keep up to **`efSearch` best nodes found so far** in a changing shortlist.

k:
when search finishes, return only the **best k** from that shortlist.

Following an edge discovers a candidate. Expanding a node means checking its neighbors. The layout is schematic; distances are the labels.

A Narrow Shortlist: efSearch = 1

First, search the graph with `efSearch`$= 1$: Results retains just **one** node. Start at A and request the nearest node ($k = 1$).

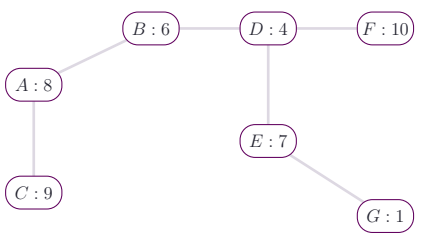
Light entries = already expanded (neighbors checked).

STATE AFTER EACH ACTION

Action	Results	Pending	Decision
Start	$[A : 8]$	$[A : 8]$	Entry node.
Expand A	$[B : 6]$	$[B : 6]$	Reject $C : 9$.
Expand B	$[D : 4]$	$[D : 4]$	D replaces B.
Expand D	$[D : 4]$	$[\,]$	Reject $E : 7, F : 10$.

Why miss G? E is farther than the retained D, so E never enters Pending. Its edge to G is never explored.

SAME GRAPH; ENTRY A



Labels are distances to q.
G exists in the graph, but is never discovered.

Stop: Pending is empty. Return D (distance 4).

Next, increase `efSearch` to 3: keeping E will let us reach G (distance 1). A wider shortlist can improve recall; it does not guarantee exact search.

A Wider Shortlist: efSearch = 3

Now increase `efSearch` to 3; keep the same graph, entry A, and $k = 1$. **Results** retains up to 3 best nodes found; **Pending** holds nodes whose neighbors are still unchecked.

Light entries = already expanded (neighbors checked).

STATE AFTER EACH ACTION

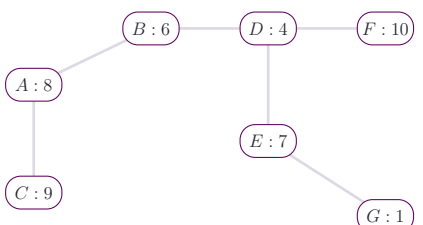
Action	Results (capacity 3)	Pending
Start	$[A:8]$	$[A:8]$
Expand A	$[B:6, A:8, C:9]$	$[B:6, C:9]$
Expand B	$[D:4, B:6, A:8]$	$[D:4, C:9]$
Expand D	$[D:4, B:6, E:7]$	$[E:7, C:9]$
Expand E	$[G:1, D:4, B:6]$	$[G:1, C:9]$
Expand G	$[G:1, D:4, B:6]$	$[C:9]$

Add an unseen neighbor to **Pending** only if **Results is not full** or the neighbor is **closer than its worst node**.

What changes? We now retain $E:7$, replacing $A:8$; expanding E discovers $G:1$. $F:10$ is still rejected.

Stop: Pending's nearest $C:9$ is worse than Results' worst $B:6$. Return G (distance 1).

SAME GRAPH; ENTRY A



Labels are distances to q.
Expand = check neighbors.
D stays in Results after expansion.

Building: Search for Candidates, Then Add Links

Now insert a new vector x, with the same distances to $A, \ldots, G$ as q. Use the same search process in a layer where x will be added.

SEARCH: `efConstruction`$= 3$

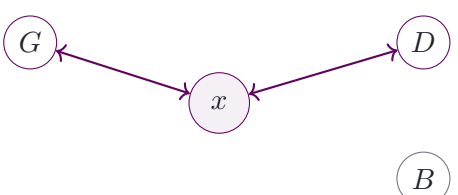
Keep at most 3 promising candidates while searching. The worked example finishes with:

$$[G : 1,\ D : 4,\ B : 6]$$

A larger shortlist allows more alternatives during construction, but costs more build time.

CONNECT: $M = 2$

Select up to 2 neighbors. Using nearest-only selection here, choose G and D.



efConstruction limits the search shortlist; M limits selected links. Larger M uses more memory. Queries use efSearch and k without adding edges.

HNSW Parameters: Effects and Tradeoffs

Separate **query-time choices** from the choices that shape the **graph we build**.

PARAMETER	CONTROLS	WHEN INCREASED	COST / TRADEOFF
`efSearch`	Query shortlist capacity	Usually higher recall: fewer missed neighbors.	More query work; higher latency.
`efConstruction`	Insertion shortlist capacity	Usually better connections; eventually, diminishing gains.	Slower graph construction. Does not set the link count.
M	Neighbor-link budget	More routes through the graph.	More stored links and construction work.
k	Number of answers returned	More returned neighbors; not an accuracy setting.	More output; choose `efSearch` $\geq k$.

Query time: change `efSearch` and k without rebuilding.
Build time: to apply new `efConstruction` or M settings throughout an existing graph, rebuild it.

CHAPTER 4

Beyond Dense Vector Search

Meaning and Exact Words Are Both Useful

Two queries can need very different kinds of evidence.

DENSE SEARCH

Query: baby dogs
Useful match: puppies

Similar meanings can match even when the words differ.

KEYWORD SEARCH

Query: BB43300
Useful match: BB43300

An exact product code matters more than a nearby meaning.

An embedding model may miss **unfamiliar terminology** or **exact identifiers**. **Hybrid search uses both signals.**

Sparse Search: Count Words, Then Weight Them

A bag-of-words vector has one coordinate per vocabulary term.

A TINY VOCABULARY

D1: “puppies eat eat”
D2: “dogs eat”

	puppies	dogs	eat	cute
D1	1	0	2	0
D2	0	1	1	0

Most vocabulary terms are absent: most coordinates are zero.

BM25: BEST MATCHING 25

Term frequency: diminishing returns
For query “eat”, D1 has 2 matches, D2 has 1. Repetition helps, but does not double the contribution.

Rarity: inverse document frequency
“Puppies” occurs in 1 of 2 documents; “eat” in both. A rare matching term carries more weight.

Length: compare with the average
For the same match count, a longer document receives less credit: it has more chances to contain the word.

BM25: From Three Factors to a Score

Q is the query; D is one candidate document. Add the contributions of query words t to score D; **larger scores rank higher**.

$$\text{score}(D,Q) = \sum_{t \in Q} \underbrace{\text{IDF}(t)}_{\text{rarity weight}} \underbrace{\frac{f(t,D)(k_1+1)}{f(t,D) + k_1\left(1 - b + b\frac{|D|}{\text{avgdl}}\right)}}_{\text{frequency with saturation and length adjustment}}$$

READ THE SYMBOLS

$f(t,D)$: occurrences of t in document D.
$|D|$: number of tokens; avgdl: average length.
N: document count; n_t: documents containing t.
$\text{IDF}(t) = \ln\left(1 + \frac{N - n_t + 0.5}{n_t + 0.5}\right)$.

k_1: larger values make repetition saturate more slowly.
b: larger values strengthen length adjustment (0 = off).
Here use $k_1 = 1.2$, $b = 0.75$.

SAME DOCUMENTS; QUERY = “EAT”

D1: “puppies eat eat”; $f = 2$, $|D1| = 3$.
D2: “dogs eat”; $f = 1$, $|D2| = 2$.
$\text{avgdl} = 2.5$; $N = n_{\text{eat}} = 2$.
$\text{IDF}(\text{eat}) = \ln(1.2) \approx 0.182$.

$\text{score}(D1) = 0.182\,\frac{2(2.2)}{2+1.38} \approx$ **0.237**
$\text{score}(D2) = 0.182\,\frac{1(2.2)}{1+1.02} \approx$ **0.199**

One Query, Two Different Rankings

`Question(id, question, answer, category, vector)` Query: **animal**

Goal: retrieve relevant questions. The `answer` values below identify the returned records.

THREE RECORDS IN THE COLLECTION

Answer	Question snippet
Elephant	The only living **mammal** in the order Proboscidea.
nose	The gavial differs from a crocodile in this feature.
Antelope	The eland is the largest species of this **animal** in Africa.

EXAMPLE RESULTS

Dense: embedding similarity
1. Elephant
2. nose
3. Antelope

BM25: keyword relevance
1. Antelope

Why can Dense find Elephant? "Mammal" is related to "animal" in meaning, even though the exact query word is absent.

Why BM25 finds Antelope: its question contains the exact token "animal". BM25 does not infer "mammal" as a synonym.

Dense order (including "nose") depends on the embedding model and full text; these snippets alone do not determine it.

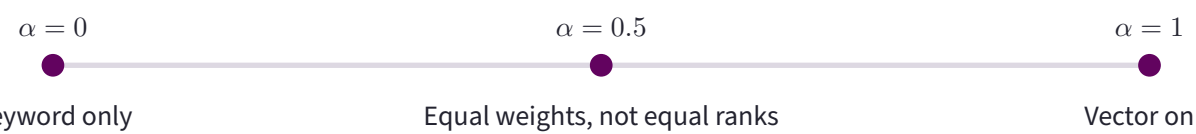
Hybrid Search: Combine Both Rankings

`Question(id, question, answer, category, vector)` Same query: **animal**

1. **Run both searches.** Collect semantic matches and keyword matches, then align results by record ID.
2. **Make scores comparable.** Use normalized similarity scores or rank-based scores; higher must mean better.

$$\underbrace{S(D)}_{\text{combined score}} = \alpha \underbrace{S_v(D)}_{\text{vector signal}} + (1-\alpha) \underbrace{S_k(D)}_{\text{keyword signal}}$$

3. Sort by the combined score.
Do not add raw BM25 scores to cosine distances: their scales and directions differ.



Possible final ranking:
Antelope → Elephant → nose.

Antelope can rise because its exact match adds a keyword signal. The final order depends on scores and fusion, not just the two rank lists.

Hybrid Search in Vector Databases

The same idea appears in many systems: **semantic retrieval + lexical retrieval + fusion**.

DATABASE	RETRIEVAL SIGNALS	HOW RESULTS ARE COMBINED
Weaviate	Dense vectors + BM25	Built-in score or rank fusion; α balances the two signals.
Milvus	Dense vectors + sparse vectors; built-in BM25 text retrieval	Built-in fusion using weights or reciprocal rank fusion (RRF).
Qdrant	Dense + sparse representations of the same records	Query API combines result lists using RRF or normalized score fusion.
Pinecone	Dense + sparse / full-text retrieval	API-dependent; document searches can be fused client-side using RRF.

RRF = reciprocal rank fusion: combine positions in ranked lists. Names link to official docs; checked November 10, 2026.

Why Semantic Search Works Across Languages

Different languages use different words for the same meaning. A **multilingual embedding model** learns to place equivalent expressions close together, for example by training on translated sentence pairs.

ONE MEANING, THREE LANGUAGES

English query
“How do I reset my password?”

German passage
“Wie setze ich mein Passwort zurück?”

Spanish passage
“¿Cómo restablezco mi contraseña?”

ONE SHARED VECTOR SPACE

Meaning matters more than language.

CHAPTER 5

Retrieval-Augmented Generation

A Confident Answer with Two Problems

A relative's child once asked me:

Which planet in our solar system has the most moons?

MY ANSWER FROM MEMORY

Jupiter: 88 moons

I recalled an old article and answered confidently, without checking a source.

WHAT WAS WRONG?

No source: I gave the child no evidence to verify the claim.

Outdated information: my recollection may no longer match the latest discoveries.

The LLM analogy: information learned earlier can produce a confident answer without evidence or up-to-date facts.

Check a Reliable Source First

What if I had checked a reputable source, such as NASA, before answering?

MY ORIGINAL RECOLLECTION

Jupiter
88 moons

AFTER A SOURCE CHECK (2023)

Saturn
146 moons

The LLM analogy: retrieve evidence before generating, rather than rely only on training.

Historical example from 2023; these counts are not current. Satellite counts change as discoveries are confirmed.

Retrieval-Augmented Generation

Add an external **content store**: public information, internal documents, or policies.

1	Retrieve	Find information relevant to the question.
2	Augment	Supply that information with the question and instructions.
3	Generate	Produce an answer grounded in the retrieved material.

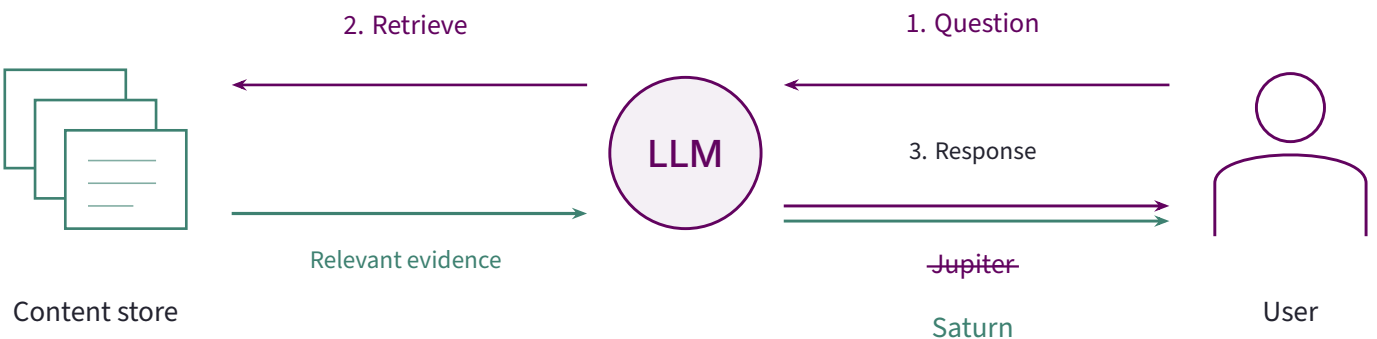
Components of a RAG System

Four components turn external information into an answer.

COMPONENT	RESPONSIBILITY	MOONS EXAMPLE
Knowledge base	Stores external documents, passage text and metadata.	NASA material about planetary moons.
Retriever	Embeds the query and finds related passages.	Fetch relevant moon information.
Integration layer	Coordinates retrieval; assembles instructions, evidence and question.	Build the augmented prompt.
Generator	Uses the augmented prompt to produce a response.	Answer Saturn; identify NASA as the source.

Optional: a **ranker** orders retrieved candidates; an **output handler** formats the response for the user.

RAG: Data, Prompt, and Response



AUGMENTED PROMPT

Instruction: answer using the supplied evidence.
Question: which planet has the most moons?
Evidence: NASA, Saturn, 146 moons (2023).

RESPONSE

Saturn, with 146 moons.
Source: NASA (2023 snapshot).

Historical comparison: the unsupported Jupiter answer is replaced by Saturn after retrieval. The application supplies the evidence.

Fresh Information, Evidence, and Uncertainty

Retrieved material helps address both outdated knowledge and unsupported answers.

FRESHNESS

Update the content store

New moons are discovered. Replace stale material; the next query retrieves the update, without retraining the LLM.

EVIDENCE

Support the answer

Use the retrieved NASA information to justify the moons answer and identify its source.

UNCERTAINTY

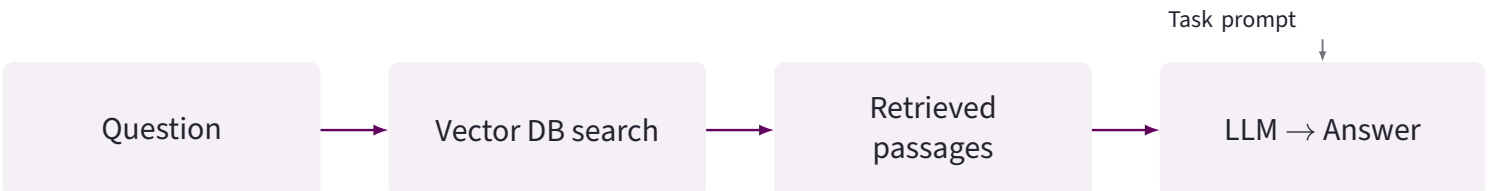
Acknowledge uncertainty

If the evidence is insufficient, say “I don’t know” instead of inventing a plausible claim.

Grounding can reduce unsupported answers; it does not guarantee accuracy or privacy.

From DB Pespective

Retrieval-Augmented Generation supplies relevant source text alongside the prompt, without retraining the LLM.



RETRIEVER + VECTOR DATABASE

Store passage embeddings and IDs. Search with the query embedding for top-k candidates; the application supplies their **text**, not just embeddings, to the LLM.

GENERATOR: THE LLM

Read the question and retrieved passages, then generate a useful answer grounded in relevant, reliable evidence.

An answer may exist in the content store but never reach the LLM. **Both retrieval and generation matter.**

RAG and Fine-Tuning

Both adapt an LLM to a domain, through different mechanisms.

ASPECT	RAG	FINE-TUNING
What changes?	The context supplied with the question.	The model's parameters, through additional training.
How is data used?	Retrieve external documents when a question arrives.	Learn from examples during training.
Typical purpose	Provide current or private information as evidence.	Adapt domain behavior and output requirements.
Updating facts	Update the external store; retrieve the new material.	To change learned parameters, run further training.

Where fine-tuning can be better: learn consistent task behavior (e.g., classification or a required JSON format). For tasks needing no external facts, it can also avoid retrieval and long evidence prompts, reducing inference latency and token use.

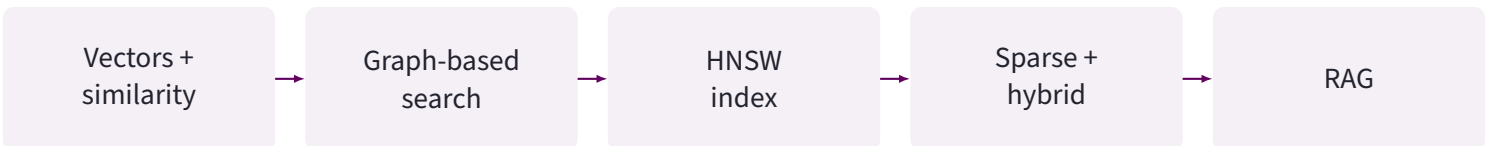
They can work together: fine-tune the desired behavior, then use RAG to supply relevant external evidence.

CHAPTER 6

Lecture Summary

From vector representations to efficient retrieval and grounded generation.

From Data to Retrieval to an Application



CHOOSE THE MATCHING SIGNAL

Meaning → dense vector search
Exact words → keyword search
Both → hybrid search
Required properties → metadata filters

KEEP THE ROLES DISTINCT

The embedding model defines the representation.
The index makes retrieval efficient.
The database manages objects and metadata.
The LLM generates from retrieved evidence.